TP3 – 3D Modelling

Drawing geometric primitives in OpenGL.

Computer Graphics
2nd year, "Ìmage and Multimedia" and "Software Engineering" tracks

Session 3

Version: v2026.1.0-rc4
(develop @ d548107 2026-03-31)

Contents

1 Objective **1**

2 Drawing primitives in OpenGL **1**

3 Exercise 1 - The Robotic Arm **2**
3.1 The code 3
3.2 Let's implement it 4

1 Objective

In this TP, we review the concepts introduced in the last TP about the OpenGL pipeline and the scene modelling, and focus on another aspect of the scene modelling: so far we used some built-in function to draw 3D objects, now we will start to see how we can build our own 3D object using some low level geometric primitives such as points and lines.

In Section 2 you find a brief introduction on how to draw geometric primitives in OpenGL, then in Section 3 you are asked to implement an interactive OpenGL program that will cover all the different aspects we saw in the last TP and the OpenGL geometric primitives.

2 Drawing primitives in OpenGL

In the last TP, we saw that GLUT provides several nice high-level objects (cones, spheres, cubes, teapots *etc.*). All these high-level objects are ultimately composed of points (or vertices) connected with lines to form the wire-frame of the object. If we want to build our own object, we then have to express it in terms of lower-level primitives, such as points, lines, triangles *etc.*

OpenGL provides several primitives, including points, lines, and polygons. Each of these objects is ultimately made up of an (ordered) collection of vertices. Vertex coordinates are specified with the OpenGL function `glVertex` (here the doc):

```
void glVertex3f(GLfloat x,  GLfloat y,  GLfloat z);
```

Vertices must be grouped to form a collection of points (or lines, or polygons, *etc.*). The beginning and end of each group is denoted by `glBegin` and `glEnd`, and the parameter passed to `glBegin` tells OpenGL how to interpret the collection of vertices that follows (here the doc). For example, the following specifies a collection of individual points (which happen to form the corners of a box)[1]:

```
glBegin(GL_POINTS);
  glVertex3f(1,1,1);
  glVertex3f(1,2,1);
  glVertex3f(2,2,1);
  glVertex3f(2,1,1);
glEnd();
```

Since `GL_POINTS` is specified, this is drawn as four individual points. Changing `glBegin`'s parameter changes the interpretation. For example, if `GL_LINES` is passed, two lines are drawn, the first having the first two points as start and end points, respectively, the second having the other two as start and end points. You can see in Figure 1 other types of primitives that are supported by OpenGL and how the list of vertices is interpreted to generate the corresponding primitive.

Finally, you can specify the current drawing color for each vertex (or primitive):

```
glBegin(GL_POINTS);
  glColor3f(1,0,0);
  glVertex3f(1,1,1);  // a red vertex
  glVertex3f(1,2,1);  // another red vertex
  glColor3f(0,0,1);
  glVertex3f(2,2,1);  // a blue vertex
  glColor3f(1,0,1);
  glVertex3f(2,1,1);  // a magenta vertex
glEnd();
```

This piece of code draws the first two points in red, the third in blue, and the last in magenta. OpenGL also provides some functions to specify the size or the width of each primitive. For example, `glLineWidth( GLfloat w);` sets the current width of the line to draw. For a point, you can use `glPointSize( GLfloat w);`. Again, since OpenGL is a state machine, the current value for the size or the width will remain in place until it is further changed.

3 Exercise 1 - The Robotic Arm

We want to implement an interactive program that allows the user to move a robotic arm. The robotic arm is composed of 3 connected joints (see Figure 2), and each of them can rotate along one axis. Each joint is sketched using a wire-frame of a parallelepiped. Moreover, to help you out with the rotation, a (local) reference system is also drawn at the base of the joint. We

[1] The indentation after `glBegin` is not necessary, but improves the readability of the code.

[git] • Branch: develop @ d548107 • Release: v2026.1.0-rc4 (2026-03-31)
Head tags: v2026.1.0-rc4

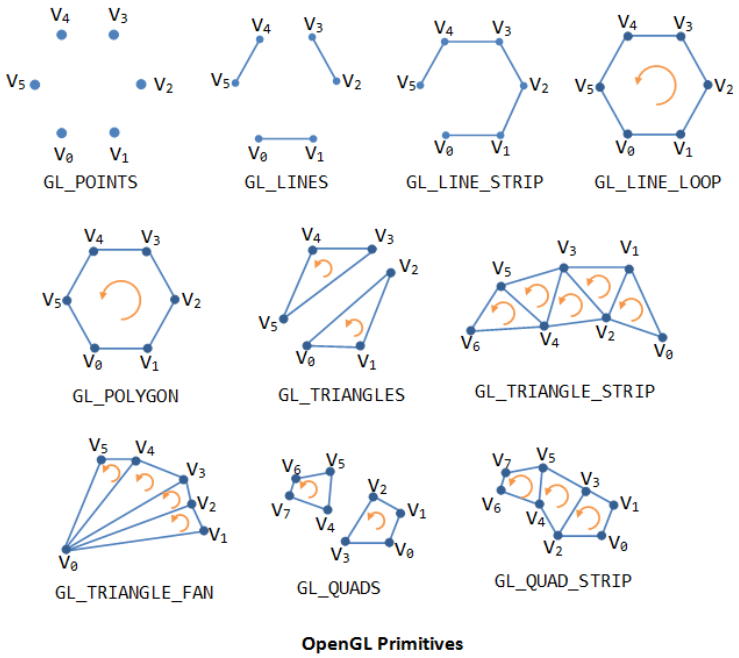


Figure 1: OpenGL geometric primitive types: the picture shows how a sequence of vertices $\{V_0, \ldots, V_n\}$ defined between `glBegin` and `glEnd` is interpreted by OpenGL, according to the type of primitive (`GL_POINTS`, ...) passed to `glBegin`.

can assume that each joint can rotate about its x-axis (the red one in Figure 2). The user can control the movement of each joint using the keyboard. In particular

- A and Z control the movement (rotation about its x-axis) of the second joint
- E and R control the movement (rotation about its x-axis) of the third joint
- the arrow keys ←, → and ↑, ↓ control the rotation of the whole arm (or, equivalently, the first joint) about its x-axis and y-axis, respectively.

3.1 The code

To implement the program, you have to fill in and complete the file `robot.c`. The code is composed as usual by a `main` routine that initializes the OpenGL's pipeline and some other functions that you should now be familiar with:

- `init` function that initializes the camera type and its position; there is nothing you have to do here, but if you want, you can change the `glLookAt` function to move the camera in case you want a different point of view.
- `reshape` callback function to set the viewport; you don't need to modify it.
- `usage` function is used to print a simple help on the screen; you don't need to modify it.
- `arrows` and `keyboard`, the callback functions to manage the user input from the keyboard; you have to complete them.

[git] • Branch: develop @ d548107 • Release: v2026.1.0-rc4 (2026-03-31)
Head tags: v2026.1.0-rc4

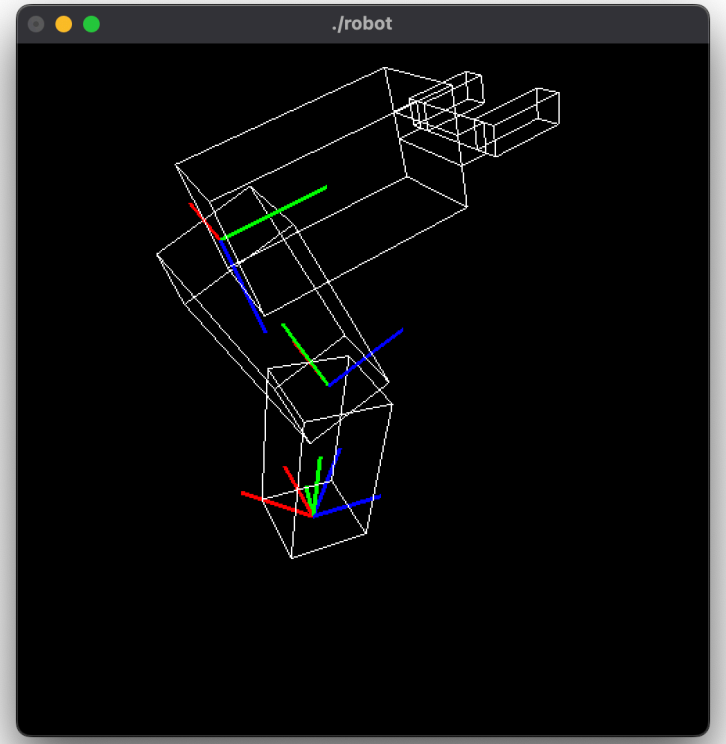


Figure 2: Example of the robotic arm.

- `display`, the display callback function; you have to complete it.
- `DrawReferenceSystem`, `DrawJoint`, and `DrawRobot` are the functions you have to implement to draw the robotic arm.

You also have 4 global variables, `Angle1`, `Angle2`, `RobotAngleX`, and `RobotAngleY`, which will be used to control the robot movements.

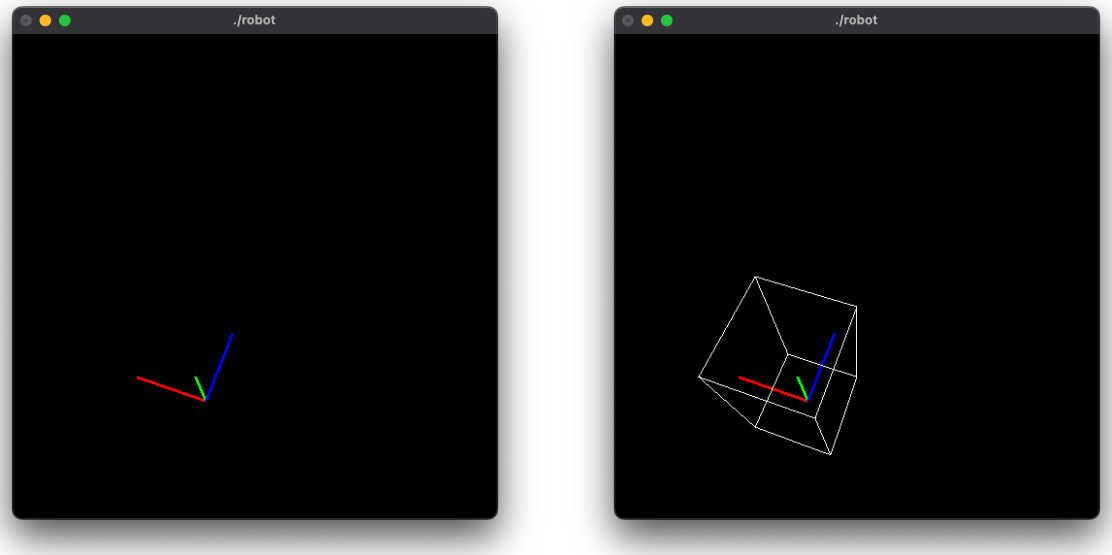
3.2 Let's implement it

The program is conceived to be highly modular so that you can implement the program incrementally and always have some visual feedback that may help you to verify and debug your code.

The whole robotic arm is composed of three joints, and each joint is composed of a parallelepiped and the reference system (see Figure 3). Therefore, we will start by developing the function `DrawReferenceSystem` that draws the reference system, then the function `DrawJoint` that draws the joint as a parallelepiped and a reference system, and finally the function `DrawRobot` that draws the whole arm as a system of 3 joints. If you implement the functions correctly at each step, you should have visual feedback of what you have coded. Finally, you will implement the part of the code that manages the input from the user and allows the robot to move.

These are the main steps you can follow:

[git] • Branch: develop @ d548107 • Release: v2026.1.0-rc4 (2026-03-31)
Head tags: v2026.1.0-rc4



(a) (b)

Figure 3: The visual output after you correctly finish to implement `DrawReferenceSystem` (a) and `DrawJoint` (b). Note that the views have been zoomed in.

1. implement `DrawReferenceSystem`: the function should draw 3 perpendicular lines corresponding to the 3 directions x, y, z (see Figure 3). Follow the comments in the code and remember that you can draw lines using the OpenGL function that has been introduced at the beginning in Section 2.
2. implement `DrawJoint`: the function should draw a parallelepiped and a reference system placed at the origin of its local reference system. To draw the parallelepiped, you can use the function `glutWireCube` and use `glScalef` to stretch it along, *e.g.*, its y-axis. Another option is to draw the parallelepiped defining its vertices using the `GL_QUAD` primitive.
   - `glutWireCube` draws the cube so that it is centered at the origin of the reference system. Hence, you have to move it to have the reference system drawn on its bottom face (here the doc).
   - Be careful when using `glScalef` (think in terms of the current modelview matrix...), you might want it only affects the local coordinate system of the parallelepiped.
3. implement `DrawRobot`: it must draw three joints, one on top of the other (on the y-axis). You can add the rotation of each joint now or later when you implement the user control.
4. Now we have the robotic arm. Complete the `display` function.
5. Complete the `keyboard` function that updates the values of the rotation angles of each joint. A reasonable update value for the angle value is $\pm 5°$, but you are free to experiment with other values. If you didn't do it before, add the relevant rotations in `DrawRobot`.
6. Complete the `arrows` function that updates the values of the rotation angles of the first joint using the arrows. If you didn't do it before, add the relevant rotations in `display`.

[git] • Branch: develop @ d548107 • Release: v2026.1.0-rc4 (2026-03-31)
Head tags: v2026.1.0-rc4

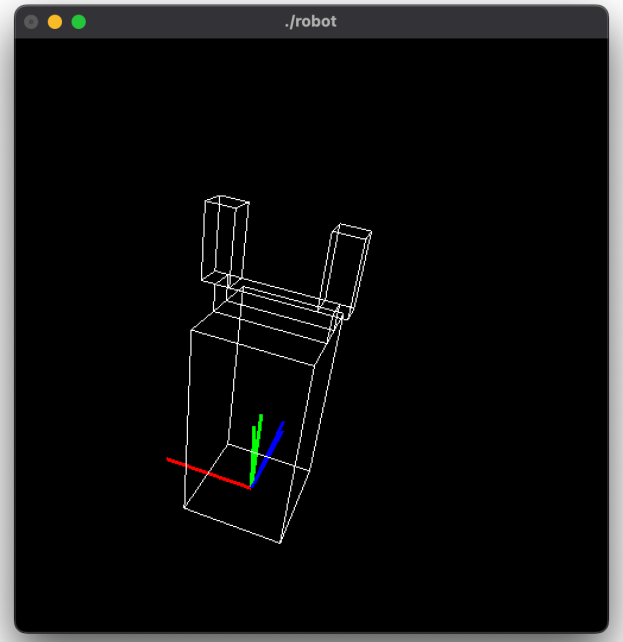


Figure 4: The last joint of the robot with a pair of pincers attached to its end.

7. Finally complete the `main` in order to register the callback for `keyboard` and `arrows`;

8. Add a pair of pincers (« pinces ») at the end of the last joint. A simple way to do it is to add two "vertical" rectangles that slide over another "horizontal" rectangle (*c.f.* Figure 4). Add two other keys to control the pincers (*e.g.* O and L), and manage the movement of the pincers so that they stop whenever they reach the limits of the opening and closing positions.

[git] • Branch: develop @ d548107 • Release: v2026.1.0-rc4 (2026-03-31)
Head tags: v2026.1.0-rc4